School Of Computing and Informatics

CG3201 Capstone Project

Capstone Project – Mid-Term Report

Muhammad Adam Danish Bin Shukri

B20230060

BSc (Hons) in Computing (Major in Software Development)

Royal Brunei Police Force

Jabatan Siasatan dan Kawalan Lalulintas

UTB Supervisors:

Dr. Hjh Wida Susanty binti Hj Suhaili

Ak. Dr. Mohd Salihin bin Pg Hj. Abdul Rahim

Organisation Supervisors:

ASP Pg Hjh Nafiah binti Hj Asli

Submission Date:

14th March 2026

Abstract

This report outlines the enhancement and expansion of an existing proof-of-concept mobile application for legal reference, developed initially by a previous attachment student using Flutter and PHP/MySQL. The project aims to transform this prototype into a production-ready, secure, and multi-platform tool for enforcement offices accessing Road Traffic Act (RTA) and Road Traffic Rules (RTR) legislation, specifically Chapter 68 and associated compoundable fines. The core problem addressed is the 'access lag' between modern field operations and outdated legal reference methods, currently static PDFs and paper documents, which results in significant inefficiencies, potential enforcement inconsistencies, and reduced public confidence. The proposed solution emphasizes three critical pillars: security hardening through enterprise-grade authentication and encryption; user-centric refinement through structured engagement with JSKLL officers; and strategic platform expansion to iOS and mobile web using Flutter's cross-platform capabilities. The project will deliver a complete application suite with administrative content management, multi-language support, and comprehensive documentation. Within a six-month timeline, this initiative will hope to equip enforcement officers with an authoritative, accessible legal tool, directly improving enforcement efficiency and laying a scalable foundation for future digital legal services.

Keyword: mobile application, law enforcement, legal technology, RTA/RTR, Flutter, digital government, user-centered design, feasibility analysis

Table of Contents

Abstract 2
1.0 Introduction 5
1.1 Refined Problem Statement 5
1.2 Project Scope 6
1.3 Boundary Conditions 7
2.0 Background Study 7
2.1 Organization Background 7
2.2 Existing System/Process 8
2.3 Affected Stakeholders 9
2.4 Literature Review/System Analysis 9
2.5 Identified Gaps/Issues 11
3.0.1 Proposed Solution 11
3.1 Methodology 11
3.2 System Design Overview 12
3.3 Tools and Technologies 12
3.3.1 React Native 13
3.3.2 Flutter 14
3.3.3 Ionic + Capacitor 14
3.4 System Architecture 17
3.5 Database Tool 18
3.5.1 MySQL 18
3.5.2 SQLite 19
3.5.3 Firebase Firestore (NoSQL) 19
3.5.4 PostgreSQL 19
4.0 Analysis 22
4.1 Requirement Analysis 22
4.2 Feasibility Analysis 23
4.3 Risk Analysis 24
5.0 Project Plan & Projection 25
5.1 Timeline 25

6.0 Conclusion 26
References 27

1.0 Introduction

With the rapid advancement of civilization within Brunei Darussalam, the nation has experienced significant growth within its infrastructure, technology and societal dynamics. As a result, it caused a vast volume of information circulating around various domains, particularly in the matter of law. Thus, a means to understand and access the legal framework is more critical than ever, particularly within the area of Road Traffic Acts.

This project aims to convert a student-developed proof-of-concept mobile application into a production-ready, secure, and multi-platform legal reference tool for enforcement officers accessing Road Traffic Act (RTA) and Road Traffic Rules (RTR) legislation. The existing prototype, built with Flutter and PHP/MySQL, successfully demonstrated core functionality but lacked enterprise-grade security, multi-platform support, and formal deployment mechanisms. The core problem addressed is the significant inefficiency in current legal reference methods, static PDFs and paper document, which creates an “access lag” between modern field operations and outdated tools.

This report documents progress during the first ten weeks of the six-month project timeline. The initial phases focused on three critical workstreams: security auditing and codebase assessment, structured user research with JSKLL officers, and foundational development of the administrative Content Management System. This report presents refined problem definitions based on early findings, updated background study incorporating lessons learned, finalized technical design decisions, detailed analysis of requirements and user feedback, and a revised project plan for the remaining work.

1.1 Refined Problem Statement

In the modern-day Brunei Darussalam, the increasing production of cars has resulted in growing numbers of road accidents occurring close to daily making it a common occurrence. Yet Royal Brunei Police Force (RBPF) or *Jabatan Pengangkutan Darat* (JPD) lack the efficiency for quick access to legal information during accidents, or even during roadblocks. Thus, the reliance on manual reference regarding the law often creates delays, inconsistencies, and frustrations for both officers and citizens.

The current prototype provides a good basis, but it is limited to an Android-only, single language academic tool with no formal deployment or upgrade options. To fully achieve it potential for boosting enforcement efficiency and public legal knowledge, the application must evolve into a secure, scalable, and easily accessible platform. The opportunity is to turn a promising prototype into a key component of digital enforcement strategy.

This would primarily affect field enforcement officers as they need a reliable, official tool on their preferred device, potentially in multiple languages. JSKLL personnel and IT departments also require a secure, maintainable, multi-platform system.

The prototype was developed without the security requirements that are required for official government tools. There is no authentication/authorization, any users can access all data, create liability issues and prevent usage tracking. The current implementation cannot support organization-wide deployment. Legal amendments require developer intervention instead of administrative control, creating dangerous delays in updating fine amounts or legal interpretations. Single-language (English) interface excludes non-native speakers and limits public accessibility; this creates a language barrier.

Through initial user research and codebase assessment, the original problem has been refined and validated. The core problem remains where enforcement officers lack an efficient, reliable mobile tool for accessing RTA/RTR legislation in the field. However, early findings have revealed additional dimensions to this problem:

1. Authentication Gap: There is a concern about using an unauthenticated app for official duties and citing accountability. The prototype's lack of user authentication creates uncertainty about who accessed what information and when.
2. Content Trust Deficit: There is an uncertainty about whether the prototype contained the most current legislation. Without a clear update mechanism and version indicator, officers cannot trust that the information they are citing is legally current, a critical liability issue.
3. Workflow Integration Needs: Beyond simple search, officers identified specific workflow pain points: needing to generate proper legal citations for reports, calculating compoundable fines accurately, accessing frequently used sections quickly during traffic stops.

1.2 Project Scope

In-scope components include codebase audit & refactoring, security implementation and administrative ecosystem which are in Phase 1: Production Hardening & Security. Codebase audit includes comprehensive security vulnerability assessment, performance optimisation and documentation of existing architecture and dependencies. Security implementation ensures there is role-based access and secure authentication with session management. Administrative ecosystem involves web-based content management system (CMS) for legal officers, version control for legislative content, user management console for IT administrators and analytics dashboard for usage monitoring.

Phase 2 involves user-centric development and enhancements. It includes structured user engagement program, workflow integration features and accessibility & localisation framework. Usability testing sessions with prototypes and workshops with stakeholder groups (RBPF and involved units). Quick-citation generation for field reports and offline-first architecture enhancement are part of the workflow integration features. Localisation (l10n) implementation (Industry Knowledge, n.d.), initial bilingual support (English +

Bahasa Melayu) and responsive design optimisation for various device sizes make up the accessibility and localisation framework scope.

Phase 3 is platform expansion and deployment which involves multi-platform deployment, deployment infrastructure and sustainability framework. For multi-platform deployment, the project is to have iOS application development and App Store deployment, progressive web app for desktop/mobile web access, consistent UX/UI across all platforms and platform specific optimization (push-notification, etc.). Deployment infrastructure involves enterprise mobile device management integration, App store deployment (Google Play, Apple App Store). Comprehensive technical documentation, administrator and user training materials, supporting ticketing system integration and roadmap for future enhancements (additional legislation, features) are part of the sustainability framework scope.

1.3 Boundary Conditions

1. Technical boundaries:
   - Maintain existing Flutter/PHP/MySQL architecture unless critical flaws are identified.
   - Support only officially released mobile OS versions (no beta/alpha support)
   - Web versions are limited to modern browsers (Edge, Chrome, Firefox)
2. Content Boundaries:
   - Legal content responsibility remains with JSKLL legal unit
   - No legal advice or interpretation features
   - Historical versioning limited to previous 2 amendments
3. Support Boundaries:
   - Training limited to “train-the-trainer” approach
   - Custom feature development beyond agreed scope requires separate agreement

2.0 Background Study

2.1 Organization Background

The ‘Pasukan Polis Diraja Brunei’ (PPDB), or the Royal Brunei Police Force (RBPF), is the primary law enforcement agency in Brunei Darussalam. Established on January 1, 1921, it operates under the jurisdiction of the Ministry of Home Affairs. The RBPF is responsible for maintaining law and order, preventing and investigating crimes, ensuring public safety, and upholding national security. I have been assigned under ‘Jabatan Siasatan dan Kawalan Lalulintas (JSKLL) under supervision of ASP Pg Hjh Nafiah binti Pg Hj Asli. She oversees ‘Unit Penyiasatan dan Penyilidikan’ (UPP).

2.2 Existing System/Process

According to previous attachment student's studies, Ak Mohd Haziq Luthfil bin Pg Shahrol Rezal Malek Faesal, data entry is currently done manually. When a roadblock is conducted, forms of 'Police 180' or 'Police 3' are given to an offender, Police 3' is given out if an offender has committed multiple offences. The RBPF faces challenges in manual processes and lacks automation in handling redundant tasks. While criminal cases investigation done by the RBPF is handled through a digital database and is somewhat automated within their application called Advanced Records Management Systems (ARMS), JSKLL lacks the system to handle these redundant tasks and would store information manually.

During the on-site roadblock, investigation officers (IO) require a reference list to refer to. Even experienced personnel would need to refer to the list occasionally, to reduce the risk of human error (wrong act, incorrect spelling, etc.).

PASUKAN POLIS DIRAJA BRUNEI
THE ROYAL BRUNEI POLICE FORCE

Polis 214

Borang No. 2
Form No. 2

TAWARAN UNTUK MENYELESAIKAN KESALAHAN-KESALAHAN DI BAWAH UNDANG-UNDANG LALULINTAS JALAN RAYA, 1954
OFFER TO COMPOUND AN OFFENCE UNDER THE ROAD TRAFFIC ACT, 1954

Apabila menjawab sila nyatakan
In reply please quote

A № 039901

Pejabat / Balai
Office / Station .....
No. Laporan
Report No. : .....

Tempat / *Place* : .....
Tarikh / *Date* : .....

Awang / Dayang,
Sir / Madam,

Satu laporan telah dibuat terhadap Awang / Dayang sebagai pemandu kenderaan bermotor ..... yang bernombor pendaftaran ..... dan menyatakan kesalahan di bawah Bab / Peraturan .....
Dari Undang-Undang Lalulintas Jalan Raya, 1956
Peraturan-Peraturan Lalulintas Jalan Raya

A report has been made against you as driver of motor vehicle ..... bearing registration number ..... alleging the following offence under Section / Regulation ..... of the Road Traffic Act, 1956
Road Traffic Regulations

Tarikh / *Date* ..... Waktu / *Time* ..... a.m. / p.m. Tempat / *Place* .....

Kesalahan-Kesalahan / *Offences* .....

2. Awang / Dayang adalah dengan ini dimaklumkan bahawa pada menjalankan kuasa-kuasa yang diberikan kepada saya oleh Bab 91 dari Undang-Undang tersebut. Saya bersedia untuk menyelesaikan kesalahan ini bagi sebanyak $..... (Ringgit) Jika tawaran ini diterima, pembayaran hendaklah dibuat dengan wang tunai atau pesanan wang di pejabat yang disebutkan di atas dan sekeping resit rasmi akan dikeluarkan kelak.

You are hereby informed that under the powers vested in me by Section 91 of the said Act, I am prepared to compound this offence for the sum of $..... (Dollars .....) If this offer is accepted payment should be made by cash or money order to the office quoted above and an official receipt will be issued.

3. Tawaran untuk menyelesaikan kesalahan ini terus berjalan kuatkuasanya selama ..... hari saja mulai dari tarikh notis ini dikeluarkan dan jika jawapan tidak diterima dalam tempoh tersebut maka tindakan saman akan diarahkan supaya dijalankan.

This offer to compound the offence remains in force for ..... days only from the date of the issue of this notice and if no reply is received within that period proceedings be summons will be instructed.

Kepada / *To* :
.....
.....

.....
Pegawai Polis atau Pengawal
Police Officer or Controller

PERINGATAN - Lesen memandu Awang / Dayang tidak akan dicatat, atau digantung atau dibatalkan disebabkan oleh kesalahan yang c
NOTE - *Your driving licenc* *reason of the offence compounded above.*

JPK 2014

Figure 1: Police 214

Police 214 (P214) is given to those who have violated the road traffic act and road traffic regulations. When given 'P214', they are given a duration of 14 days to pay the given penalty depending on which of the road traffic rules have been violated. If the offender fails to pay the penalty, they will be required to go to court.

Police 180 is given to offenders who are to be summoned to the police station. A maximum of 3 issues of P180 can be given. If the offender fails to pay the penalty, the offender will be given Police 3.

Police 3 is used when the offender has failed to make the payment for the compoundable offence within the 14-day period given. Police 3 is used when the offender is summoned to court.

2.3 Affected Stakeholders

After the completion of the system, there will be 3 categories of stakeholder who are directly and indirectly involved with the implementation of the system.

The first stakeholder would be RBPF, who are responsible for detaining drivers who violate road traffic regulations. The process of detaining offenders is often time-consuming due to the need to go through the physical rules and regulations booklet to confirm their offence. With the implementation of the system, it would streamline the procedure significantly, allowing officers to efficiently present evidence of the violation to the offenders, which reduces delays. RBPF administrators will have privileges such as adding new laws or editing existing ones if any changes are added in the future.

Secondly, *Jabatan Pengangkutan Darat (*JPD), whose role includes incorporating the enforcement of laws into the education curriculum for students obtaining their driver's licenses. This ensure that new drivers are well-informed about road safety and traffic regulations early on, all of which can be done efficiently and digitally via implementing this system.

Lastly, the public, though indirectly involved, can benefit from the project by gaining increased exposure to traffic laws. By implementing this system, it can allow awareness to be spread across the country regarding the law. With the availability of the application, users will have easy access to information about regulations each time they interact with the system.

2.4 Literature Review/System Analysis

Evaluating the React Native Framework for Cross-Platform Mobile Development

The rapid growth of mobile applications has created a significant challenge: supporting both Android and iOS require developing two separate native codebases, which is time-consuming, costly, and demands expertise in different language (Java/Kotlin for Android, Swift/Objective-C for iOS). This challenge has driven the search for cross-platform solutions that can deliver native-like experiences without duplicated effort (Danielsson, 2016).

Prior to React Native, several approaches existed with notable trade-offs. Responsive web applications adapt to screen sizes using HTML5/CSS3 but lack native device integration. Hybrid frameworks (e.g., Ionic) wrap web code in a native WebView, reducing development costs but struggling to match native performance and user experience. Native development remains the benchmark, offering optimal performance through platform-specific languages and APIs, but requires separate codebases.

React introduced a declarative programming style for building user interfaces. Its key innovation is the Virtual DOM, an in-memory representation of the actual DOM that enables efficient updates through reconciliation, minimising direct manipulation. Building on React, React Native operates on a “learn once, write anywhere” philosophy. Unlike hybrid solutions, it is not WebView-based; instead, it compiles React code to real native components via a “bridge” enabling asynchronous communication between a JavaScript thread and native APIs. Layout executes on a separate thread, keeping the UI responsive.

Danielsson’s thesis synthesises established evaluation approaches. Performance benchmarking measures CPU load, memory usage, power consumption, and execution time using tools like Trepn Profiler. Feature-based models like FURPS (Functionality, Usability, Reliability, Performance, Supportability) provide structured qualitative comparison. Comparative user studies examine look, feel, and responsiveness. Danielsson employed expert user evaluation with IT consultants who could identify subtle deviations from native behaviour, providing critical assessment grounded in usability principles.

In conclusion, Danielsson’s thesis provides and early comprehensive evaluation concluding that React Native presents a compelling value proposition. It significantly streamlines development, enables shared codebases, and delivers user experience remarkably close to native, with performance penalties that are often imperceptible or acceptable to users. Key areas for improvement include framework stability and first-party component development. For many applications, React Native offers a viable alternative to maintaining separate native codebases, successfully balancing development efficiency with user experience quality.

2.5 Identified Gaps/Issues

1. **Accessibility Gap:** Android-only deployment excludes iOS users and limits future public web access.
2. **Language Gap:** Single-language support hinders usability in multilingual contexts.
3. **User-Experience Gap:** While initial feedback was incorporated, deeper, structured engagement is needed to uncover advanced needs and workflow integrations.
4. **Deployment & Maintenance Gap:** No pathway for official distribution, updates, or content management.

3.0 Proposed Solution

Currently, the authorities who are responsible for the Road Traffic Acts (RTA) are mainly RBPF apart from JPD on handling specialized vehicles (e.g., Lorries). Within Royal Brunei Police Force (RBPF), the Department of Traffic Control and Investigation (JSKLL), are tasked to charge offenders who are guilty of violating the RTA. Currently, their method of citation and referencing the violation to the offender is through paper based references. With the number of Acts within the Road Traffic Acts lawbook, it wouldn't be a surprise if even seasoned officers cannot remember all the laws and it would create delays when trying to cite offenders when officers must go through the Chapter 68: Road Traffic Act lawbook. There is also the risk of inconsistencies and human errors, where officers could cite the wrong section and mismatch fines. For this reason, I justify the reasoning of the previous attachment student, Ak Mohd Haziq Luthfil Al'Hafez, project to develop a proof on concept of an application that allows the stakeholders to be able to search up the Road Traffic Acts and Road Traffic Regulations library efficiently.

3.1 Methodology

For this project, I will be utilising the hybrid agile-waterfall method approach (Kahsyap, 2024). By applying Waterfall principles to the core infrastructure (database, schema), comprehensive documentation is produced upfront. This satisfies legal and compliance requirements before a single line of user interface code is written. This approach isolates flexibility to the user-facing layers. The front end can be developed in Agile sprints, allowing for the same user-centric adaptability as Pure Agile, but without destabilising the compliant backend. This model is superior for risk mitigation. The Waterfall component ensures the 'foundation' (data layer) is solid. The Agile component manages the risk of building an unusable product by iterating based on user testing. While the upfront planning phase takes slightly longer, the backend is built correctly once. Subsequent Agile sprints can then deliver frontend features rapidly without the risk of breaking core compliance.

3.2 System Design Overview

The proposed solution builds directly upon the existing previous student's prototype while addressing the critical gaps identified through security auditing and user research. Rather than rebuilding the application from scratch, we are taking a strategic approach that preserves the valuable work already completed while adding the enterprise-grade features necessary for official deployment. The system architecture follows a modern, layered design that separates concerns, enhances security, and enables efficient development across multiple platforms.

At its core, the system consists of three main layers working together seamlessly. The client layer, built with Flutter, delivers the application to Android, iOS, and web users from a single codebase. The API gateway layer, powered by Laravel, handles authentication, request validation, and routing to appropriate services. The data layer, comprising MySQL for primary storage, ensures reliable and fast access to legislative content. This separation of responsibilities makes the system more maintainable, scalable, and secure than the original prototype, where these concerns were not clearly delineated.

The decision to retain and enhance the existing Flutter codebase rather than rewrite it in native technologies was made after careful consideration of several factors. First, the prototype already demonstrates that Flutter can deliver a smooth, responsive user experience for legal search and browsing. Second, maintaining a single codebase for Android, iOS, and web deployment reduces development time by an estimated 40 percent compared to building three separate native applications. Third, Flutter's growing maturity and strong community support mean that security updates and performance improvements are readily available. The alternative approach of building native Android and iOS apps separately would have required significantly more time and resources, potentially delaying the project beyond the six-month timeline.

3.3 Tools and Technologies

As the application is required to be accessible across multiple platforms: iOS devices, Android devices and web browsers, developing three separate native applications would be time-consuming and resource-intensive for this project. A cross-platform framework is recommended. Although there are multiple frameworks to choose from, evaluating the leading cross-platform frameworks against project-specific criteria will justify the final selection.

Criteria	Description	Importance
Platform Coverage	Ability to run Android, iOS and web from a single codebase	The application must be accessible on all three platforms without maintaining separate codebase
Performance	Rendering speed, animation, smoothness, and responsiveness	Legal professionals expect professional-grade performance when searching databases
UI Consistency	Ability to maintain visual uniformity across platforms.	Users should have a consistent experience regardless of device
Development Efficiency	Learning curve, tooling quality, and code reusability	As a solo developer with a fixed timeline, efficiency is critical
Ecosystem Maturity	Availability of libraries, documentation, and community support	Common features (database, HTTP requests, state management) should not require building from scratch

Table 1: Framework Evaluation Criteria

The table above establishes clear, objective criteria against which all framework options will be assessed on. This ensures that each framework was evaluated against the same standards (Lovable Team, 2026).

Three frameworks were considered for this project:

3.3.1 React Native

React Native is a JavaScript framework developed by Meta that renders using native platform components. It has a large ecosystem, JavaScript/React familiarity and strong mobile support. It’s web support relies on community projects.

3.3.2 Flutter

Flutter is a UI toolkit developed by Google that compiles to native code and renders using its own graphics engine. It has an official multi-platform support, high-performance and visual consistency. There is a steep learning curve for Dart which is Flutter's code language and its web app bundle sizes are larger.

3.3.3 Ionic + Capacitor

It is a hybrid approach that wraps web applications (HTML, CSS, JavaScript) in a native container. It uses web technologies and has a large ecosystem of web developers. Though, it has performance limitations and has a less native feel due to UI consistency challenges.

Criterion	React Native	Flutter	Ionic
Platform Coverage	Android, iOS (official); Web (community)	Android, iOS, Web (official)	Android, iOS, Web (web-first)
Performance	Moderate	High	Moderate
UI Consistency	Moderate (relies on native components)	High (custom rendering engine)	Low (web components styled to match)
Development Efficiency	High (JavaScript/React)	High (custom rendering engine)	Low (web components styled to match)
Code Reusability	High for logic; UI needs platform tuning	Very high	Very high (web code runs everywhere)
Ecosystem Maturity	Very large (npm)	Large and Growing (pub.dev)	Large (web ecosystem)
Learning curve	Shallow (for web developers)	Moderate (Dart, Flutter plugins)	Shallow (HTML/CSS/JS)

Table 2: Comparative Analysis

The comparison of the frameworks against the established criteria revealed some trade-offs that directly impact their suitability for the application.

React Native offers strong mobile support and a vast JavaScript ecosystem, making it appealing for web developers. However, there are a couple of issues with this framework. Firstly, web support relies on a community-maintained project rather than official framework support, including risk for a project requiring simultaneous mobile and web delivery. Second, the JavaScript bridge architecture creates performance overhead that could affect the smooth rendering of document viewers and search interfaces expected by law enforcement.

Ionic takes a web-first approach, allowing maximum code reuse with familiar HTML, CSS, and JavaScript. This makes development accessible, but the reliance on WebView rendering means the application never achieves true native performance or feel. For a professional tool where user experience directly impacts adoption, the trade-off in polish and responsiveness was considered unacceptable.

Flutter distinguished itself by meeting all the criteria effectively. It offers official, production-ready support for Android, iOS, and web from a single codebase, eliminating the platform coverage concerns of other frameworks. Its ahead-of-time compilation and custom rendering engine deliver the high performance required for data-intensive legal research tools. Because Flutter renders its own widgets rather than relying on native components, the application maintains visual consistency across all devices, ensuring user encounter the same interface whether on a web browser, iPad or Android tablet. The framework also achieves a high code reusability rate across platforms, maximising development efficiency. With Flutter, the need to learn Dart and larger web bundle sizes prove to be the primary drawbacks but were considered manageable. Dart is intentionally designed to be familiar to developers with Java, C#, or JavaScript experience, and web bundle sizes can be optimized through code splitting and lazy loading techniques. These trade-offs are acceptable given the framework's significant advantages in platform coverage, performance, and consistency.

After evaluating React Native, Flutter and Ionic, Flutter emerges as the optimal choice for the Law Library Application. Its combination of true multi-platform support, high performance, visual consistency, and development efficiency aligns perfectly with the requirements of delivering a professional legal research tool.

3.4 System Architecture

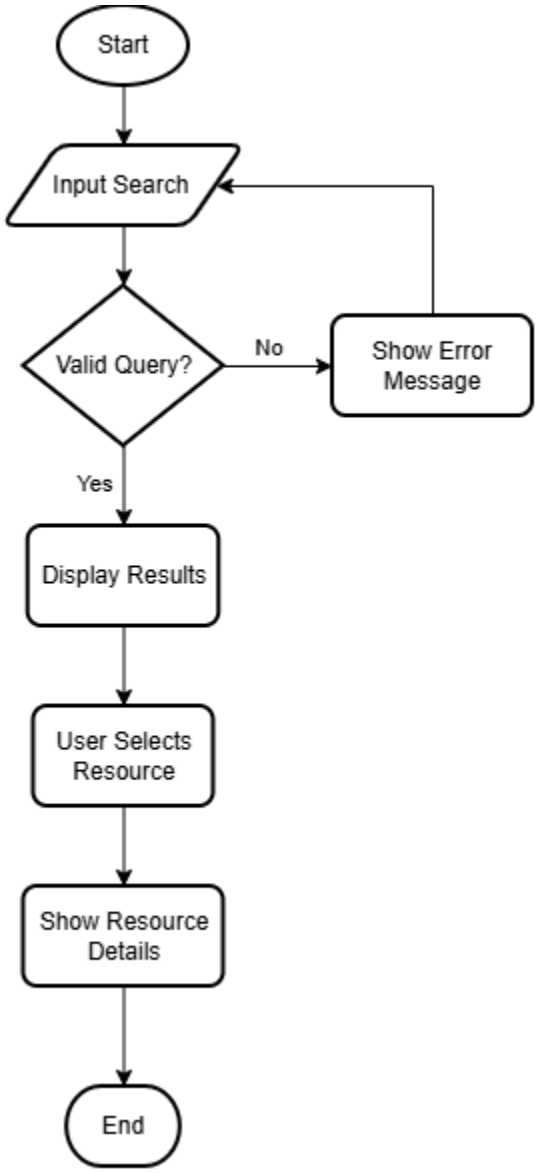

Figure 2: Search Engine Flowchart

The flowchart above illustrates how users interact with the Law Library Application. It starts with the user inputting a search query, which is then processed by the search controller. If the search if successful, the system displays the relevant queries. Users can then select the desired query which the system will then display the result in detail. In case the search query from the user is not found within the database, the system will then

show an error message to the user, indicating the query is not valid and return the user to the search tab.

3.5 Database Tool

Criterion	Definition	Importance
Query Speed	How quickly the database returns the search results	Officers need instant answers during traffic stops
Data Integrity	Accuracy and consistency of stored data	Incorrect offence information could lead to wrongful citations
Offline Functionality	Ability to operate without internet connection	Roadblocks and rural areas may have poor connectivity
Data Structure	Ability to handle structured data with define relationships	Offence data has clear relationships
ACID Compliance	Support for Atomicity, Consistency, Isolation, and Durability	Ensures data updates (new RTA/RTR, penalty changes) are applied reliably
Ecosystem Maturity	Availability of tools, documentation, and community support	A developer needs reliable resources for troubleshooting
Integration	Ease of connecting with Flutter application	Must work seamlessly with the chosen technology stack
Security	Protection of sensitive legal data	Prevents unauthorised modification of data
Cost	Licensing and hosting expenses	University final year project with limited budget

Table 3: Database Evaluation Criteria

The table above shows the criteria for assessing each database options. This ensures each database option is assessed according to objective based criteria (Li, 2025).

Four database options were considered for this project:

3.5.1 MySQL

MySQL is designed for speed and reliability, fully adhering to SQL standards. Unlike SQLite, MySQL operates with a separate daemon process that applications use to access the database. This structure allows better control over who can access the database. MySQL offers several key benefits such as popularity and ease of use, security,

performance, and replication. MySQL has its limitations as well such as licensing and proprietary features, and slow development.

3.5.2 SQLite

SQLite is often referred to as a “serverless” database. Unlike most relational database engines that run as server process and communicate with applications through inter-process communication, SQLite allows any process that accesses the database to directly read and write the database disk file. This greatly simplifies its use, as it functions almost like a file, requiring minimal configuration. SQLite’s key advantages are having a small footprint meaning it is lightweight and self-contained, user-friendly where it doesn’t require server processes, restarts, or configuration files, and portability where unlike other databases that store data in multiple files, SQLite stores the entire database in a single file. The drawbacks of SQLite are being limited concurrency where while multiple processes can read and query an SQLite database, only one process can write to the database at any given time, SQLite also doesn’t support user roles or permissions, and SQLite has security issues as server-based engines like MySQL or PostgreSQL offer better protection against application vulnerabilities and allow for more precise control over data access with fine-grained locking and better concurrency.

3.5.3 Firebase Firestore (NoSQL)

A cloud-based database platform designed for the development of web and mobile applications, helping to simplify the development process. It provides a set of tools aimed at four main objectives: development, growth, monetisation, and data analysis. Firebase stands out for its ease of use and integration into applications. Its intuitive interface and detailed documentation for all platforms make it an accessible solution for all types of developers. An important aspect of Firebase is its free tier, which includes a substantial set of services, making it an ideal option for all types of projects, from the most basic to advanced ones. Firebase is compatible with mobile applications (Android and iOS), web applications (providing SDKs for JavaScript), and video games (Unity and C++), and it also supports Flutter. Given Firebase’s closed-source architecture, app developers’ control over the platform is highly restricted. Firebase users are locked in on the platform using Firebase for app development projects which leads to many developers seeking alternative platforms for app development simply because they cannot move their apps to other platforms when the need arises (Stevenson, 2018).

3.5.4 PostgreSQL

PostgreSQL is known as “the most advanced open-source relational database”. Its design is focused on scalability and strict SQL compliance. Postgres handles multiple tasks concurrently and can perform operations without locking reads. The advantages of PostgreSQL include SQL compliance; Postgres strictly adheres to SQL standards, making it a strong choice for applications requiring full SQL compliance, open source and

community-driven; PostgreSQL is fully open-source, with a large community contributing to its development and providing ample documentation and support, and scalability where users can extend PostgreSQL programmatically and dynamically load extensions to add new functionality. The disadvantages of PostgreSQL are affecting memory performance, where each client connection in Postgres spawns a new process, which can consume up to 10MB of memory per process; and although growing, PostgreSQL's popularity still falls behind MySQL, meaning third-party tools and experienced administrators may be less available.

Criteria	MySQL	SQLite	Firebase	PostgreSQL
Query Speed	Very fast (indexed searches)	Very fast (local device)	Fast (cloud + local cache)	Very fast
Data Integrity	Strong (ACID compliant)	Strong (ACID compliant)	Moderate (eventual consistency)	Strong (ACID compliant)
Offline Functionality	Requires backend sync strategy	Excellent (native offline)	Good (offline persistence)	Requires backend sync strategy)
Data Structure	Strong (relational)	Strong (relational)	Weak (document-based)	Strong (relational + object)
Integration with Flutter	Good (via REST API)	Excellent (direct via sqflite)	Excellent (native SDK)	Good (via REST API)
Setup Complexity	Moderate (requires backend)	Simple (embedded)	Simple (cloud setup)	Moderate
Security Control	Full control	Device-level only	Managed by Google	Full control

Cost	Free	Free	Free tier/ Usage-based	Free

Table 4: Database Comparative Analysis

The evaluation of the four database options reveals distinct trade-offs that directly impact their suitability for a law enforcement mobile application (Yigal, 2018).

PostgreSQL is a powerful, feature-rich database with excellent capabilities. However, for this specific use case, it offers little advantage over MySQL while introducing slightly more complexity. Given that both are server-based relational databases requiring a backend API, MySQL's simpler ecosystem and larger community make it the more practical choice between the two.

Firebase Firestore offers compelling features for mobile applications, particularly its real-time synchronization and built-in offline support. The ability to sync data automatically when connectivity returns is valuable for officers working in areas with intermittent coverage. However, Firestore's document-based model is less suited to the highly relational nature of offence data. Queries that involve joining offences with penalties, vehicle types, and jurisdictions become more complex and less efficient in a NoSQL database.

SQLite presents a strong alternative because it runs directly on the mobile device, enabling instant offline access without relying on network connectivity. This is particularly valuable for law enforcement officers who may be in remote areas or experience network congestion during major operations. SQLite also integrates seamlessly with Flutter through the sqflite package, simplifying development. However, SQLite alone cannot solve the data distribution problem: when new offences are added or penalties change, the local database on every officer's device must be updated. This requires a synchronization strategy, effectively reintroducing a backend component.

MySQL emerges as the optimal choice when considering the complete system architecture. While it requires a backend service to serve data to mobile clients, this architecture provides several critical advantages for law enforcement applications:

First, a centralized MySQL database ensures data consistency across all users. When legislation changes and penalties are updated, every user retrieves the updated information from the central source immediately. There is no risk of users using outdated offence data because their local cache missed an update.

Secondly, MySQL's relational model perfectly matches the structured nature of legal data. Offences, penalties, vehicle classifications, and jurisdictions map naturally to tables with

foreign key relationships. Complex queries like “retrieve all traffic offences applicable to motorcycles with fines under $300” execute efficiently using SQL’s join capabilities.

Third, MySQL’s mature ecosystem provides robust tools for backup, security, and administration. Legal data must be protected against unauthorised modification and loss. MySQL’s user privilege system, encryption options, and proven backup tools meet these requirements.

Fourth, query performance for the types of lookups officers need, searching by offence code, keyword search in descriptions, filtering by vehicle type, is excellent when appropriate indexes are applied.

The primary challenge with MySQL is offline functionality. However, it can be worked around through a caching strategy. The mobile app can maintain a local copy of the most commonly accessed offence data using SQLite as cache. When offline, the app queries the local cache. This hybrid approach combines MySQL’s centralised authority with SQLite’s offline capabilities.

After systematically evaluating the potential databases against project-specific criteria, I have selected MySQL is selected as the optimal database solution for the Road Offence Act Mobile Application. Its relational model aligns naturally with structured legal data, ACID compliance ensures data integrity critical for law enforcement, and mature ecosystem provides reliable support. The offline challenge is effectively addressed through a local SQLite cache that enables field officers to access offence information anywhere, regardless of connectivity. This hybrid database architecture ensures that officers always have access to accurate, up-to-date legal information when they need it most.

4.0 Analysis

4.1 Requirement Analysis

The functional requirements are prioritised by making use of the MoSCoW method. This method is a prioritisation technique used in a project management and software development to align stakeholders on the importance of requirements. It categorises initiatives into four parts: Must-have (critical), Should-have (high value), Could-have (desirable), and Won’t-have (deferred). This framework helps focus on delivering a Minimum Viable Product (MVP) and managing objectives effectively (Villaumbrosia, 2025).

MUST HAVE	SHOULD HAVE
1. Authentication & Authorisation 2. Enhanced Search Capabilities 3. Content Management System 4. Multi-platform Support 5. Offline-first Architecture	1. Multi-language support 2. Advanced Search Features
COULD HAVE	**WON'T HAVE**
1. Integration Capabilities 2. Training & Support Features	1. Artificial Intelligence 2. Direct integration with case management systems 3. Hardware-specific features

4.2 Feasibility Analysis

The proposed enhancement of the existing proof-of-concept application demonstrates strong technical feasibility based on multiple converging factors. The foundation provided by the prototype significantly reduces technical risk, as core functionality including search algorithms, data structures, and user interface patterns has already been validated in a real-world context. The technology stack, Flutter for cross-platform frontend development, PHP/Laravel for backend services and MySQL for data management, represent mature, widely adopted technologies with extensive documentation, established best practices, and readily available developer expertise in the local market. Flutter's inherent capability for compiling to Android, iOS, and web targets from a single codebase directly addresses the requirement for multi-platform accessibility, eliminating the need for separate development teams and ensuring feature consistency across deployment targets.

Critical technical requirements such as offline functionality, full-text search, and secure data synchronisation are well-supported by existing libraries and frameworks within this ecosystem. For instance, Flutter's robust plugin architecture offers proven solutions for local data storage (SQFLite) and background synchronisation, while MySQL 8.0's enhanced full-text search capabilities with Boolean operators and ranking meets the complex query needs of legal terminology. The integration of a web-based Content Management System using proven frameworks like as Vue.js or React with the existing PHP backend follows standard patterns that are consistent with current IT department

competencies. The project's modular strategy helps to reduce the key technical obstacles, which include guaranteeing consistent performance across all platforms, providing strong conflict resolution for offline data synchronisation, and achieving the strict security certification required for government systems. This enables focused attention to each difficult area, early prototyping of synchronisation logic, and parallel security review processes that can run alongside feature development. The current government IT infrastructure provides a compliant hosting environment, and the prototype's architecture illustrates strong foundational decisions in data modelling and API design, which adds to technical viability.

Operational feasibility is very high due to strong user demand and established organisational collaborations. The student project was created in close collaboration with JSKLL personnel, laying a solid foundation for stakeholder engagement and demonstrating the main concept's usefulness in field operations. This pre-existing relationship provides access to user groups for research, testing, and feedback throughout the enhancement process, significantly boosting the likelihood of user acceptance and satisfaction. The shown pain point, sluggish PDF-based and physical books legal lookup, is a widely acknowledged issue among enforcement personnel, providing motivation to implement a better solution. The suggested system is precisely aligned with broader governmental digital transformation goals, positioning it for favourable resource allocation and policy support.

From an organizational standpoint, the change is manageable. The program improves, rather than replaces, current core systems, minimizing impact to established workflows. Training requirements are minimal due to the straightforward design of mobile applications and the experience many officers have with the prototype; a train-the-trainer methodology can rapidly cascade knowledge throughout the department. The IT support organization can adapt to keep the application running because the technological stack corresponds with existing departmental expertise, and extensive documentation will be a major deliverable. Potential opposition to change, a common operational risk, is minimized by the visible time-saving gains already seen in prototype use, as well as by including officers as co-designers in the enhancement process. The operational model, centered on a centralized CMS managed by the legal team with automatic deployment to officer devices, establishes a sustainable cycle for maintaining legal accuracy without burdening the IT department with routine content updates.

4.3 Risk Analysis

1. Risk:
    a. The law is always changing; therefore, the application might need to be updated annually. If not, it might risk in false penalty amount or violated law turned out to be altered.

b. Time scope; the project duration, spanning over 5 months for development and implementation, shows a critical issue in terms of completing all the planned activities and deliverables within the allocated time frame. It can lead to rushed development, quality reduction or unable to meet the deadline.
c. Lack of expertise; the lack of skill to learn something new such as new framework (Flutter), programming language (Dart), etc., may lead to investing additional time into self-training to learn the skill. The learning curve may impact the ability to meet the project deadline.

5.0 Project Plan & Projection

The project will be executed within a six-month timeframe, employing a parallelised, agile approach with three overlapping workstreams to meet this constraint. The plan begins with a critical Foundation Sprint (Month 1), where workstreams for security hardening (implementing authentication, encrypting, and secure APIs) and User Research & Road Mapping (conducting focused meetings with JSKLL officers to finalise the feature priority list) will run concurrently with an initial code audit of the existing prototype. This ensures the development is immediately guided by both technical and user requirements.

5.1 Timeline

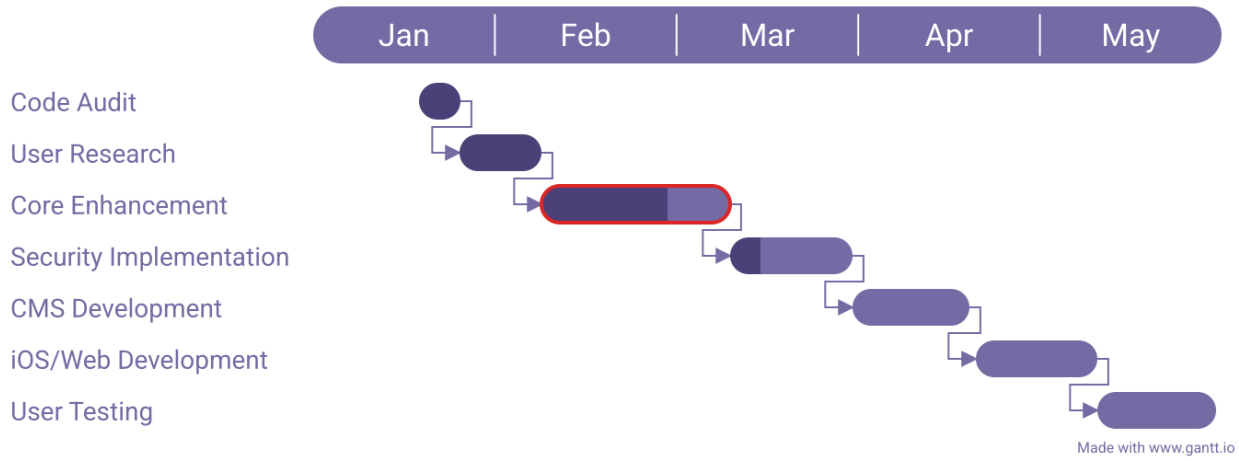

Figure 3: Gantt Chart Timeline

Based on the Gantt chart provided, the projected has successfully completed its initial phases and is progressing according to the six-month timeline. The visual representation of the project schedule clearly shows the parallel workstreams and dependencies that have been carefully managed to ensure timely delivery. Main features such as the search

functionality and favourites page fall under ‘Core Enhancement’ which is still in progress and will take a total of 30 days to complete, being 20 days for the search feature and the rest for the favourites page. Lighter features such as ‘iOS/Web Development’ will take a significantly less time duration as my main focus is the advanced search features. After the development of the system has been completed, a user testing phase is done to test the limitations, errors, and flaws of the app when testing.

6.0 Conclusion

The project has progressed successfully through its first few stages, achieving the key milestones. The completion of this project will affect how law enforcements do their job positively. The impact upon successful implementation of the system will help the law enforcement gain a reliable, official tool that dramatically speeds up legal lookup times, directly translating to greater efficiency and consistency in the field. The inclusion of iOS and web platforms ensures equal access for all personnel, while the administrative CMS empowers the legal team to maintain accuracy.

References

Ahto K, P. J. (2011). *Modernization of The E-Government in Estonia.*

Armand Krasniqi, M. K. (2023, June 22). *Challenges and Perspectives of Law and Digitalization: Case Study of Contract Law.* Retrieved from Learn & Teach: https://www.researchgate.net/profile/Mersiha-Kacamakovic-2/publication/374020037_CHALLENGES_AND_PERSPECTIVES_OF_LAW_AND_DIGITALIZATION_CASE_STUDY_OF_CONTRACT_LAW/links/67f6e32ee8041142a1701a84/CHALLENGES-AND-PERSPECTIVES-OF-LAW-AND-DIGITALIZATION-CASE-STU

Bjork. (2002). *The Impact of Electronic Document Management on Construction Information Management.*

Danielsson, W. (2016, September 23). *React Native application development: A comparison between native Android and React Native.* Retrieved from Digitala Vetenskapliga Arkivet: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-131645

Day, B. (2025, june 2). *The Ultimate Guide to S.M.A.R.T. Goals*. Retrieved from Forbes: https://www.forbes.com/advisor/business/smart-goals/

Industry Knowledge. (n.d.). *WhP*. Retrieved from WhP International: https://whpintl.com/blog/l10n-t9n-i18n-or-g11n-a-new-language/

Kahsyap, S. (2024, April 15). *What is hybrid project management?* Retrieved from ProofHub: https://www.proofhub.com/articles/hybrid-project-management

Li, J. (2025, January 14). *SQLite vs MySQL vs PostgreSQL: A Comprehensive Comparison*. Retrieved from Medium: https://medium.com/chat2db/sqlite-vs-mysql-vs-postgresql-a-comprehensive-comparison-6e5fc33ecc03

López Jiménez, D. D. (2022). *The Trusted Third Party or Digital Notary in Spain.* Retrieved from International Review of Law, Computers & Technology.

Lovable Team. (2026, January 13). *Best Frameworks for Mobile App Development*. Retrieved from Lovable: https://lovable.dev/guides/best-framework-for-mobile-app-development

Stevenson, D. (2018, September 25). *What is Firebase? The complete story, abridged*. Retrieved from Medium: https://medium.com/firebase-developers/what-is-firebase-the-complete-story-abridged-bcc730c5f2c0

Team Counterpoint. (2025, November 24). *Counterpoint*. Retrieved from counterpointresearch: https://counterpointresearch.com/en/insights/global-smartphone-os-market-share

Villaumbrosia, C. G. (2025, January 29). *MoSCoW Prioritization: How to Use It in Product Management*. Retrieved from Product School: https://productschool.com/blog/product-strategy/moscow-prioritization

WCAG 101: Understanding the Web Content Accessibility Guidelines. (2024, October 9). Retrieved from WCAG: https://www.wcag.com/resource/what-is-wcag/#:~:text=WCAG%202.1%20was%20published%20in,%2C%20hearing%2C%20and%20cognitive%20disabilities.

Yigal, A. (2018, November 8). *Sqlite vs. MySQL vs. PostgreSQL: A Comparison of Relational Databases* . Retrieved from logz.io: https://logz.io/blog/relational-database-comparison/

Żurawiecki, J. (2024, April 29). *Key Differences between Waterfall, Agile, and Hybrid*. Retrieved from BigPicture: https://bigpicture.one/blog/waterfall-agile-hybrid-differences/